

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	C.Sandeep	Reg. No.:	192472368
Section / Batch:	AI / 2024	Date:	

Code of Conduct

I C.Sandeep (192472368) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: C.Sandeep

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	3	3	3	3	3	15	78
2	3	3	3	3	3	15	
3	4	3	4	3	4	18	
4	3	3	3	3	3	15	
5	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
Dr. T. Kumaragurubaran		
Signature: Kumaragurubaran T	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Aim

To implement a **Hidden Markov Model (HMM)** for **Part-of-Speech (POS) Tagging** using the **Viterbi Algorithm** and predict the grammatical category of each word in a given sentence based on transition and emission probabilities.

Objective

- To understand the concept of Hidden Markov Models.
- To implement POS tagging using supervised learning.
- To compute transition and emission probabilities.
- To apply the Viterbi Algorithm for finding the most probable sequence of tags.
- To classify words into grammatical categories such as Noun, Verb, Pronoun, and Determiner.

Theory

Part-of-Speech (POS) Tagging is one of the fundamental tasks in Natural Language Processing (NLP). It assigns a grammatical label to each word in a sentence.

Examples of POS tags include:

Word POS Tag

I PRON

eat VERB

rice NOUN

the DET

A **Hidden Markov Model (HMM)** is a statistical model used to determine the most probable sequence of hidden states (POS tags) from observable data (words).

The HMM consists of:

- **States:** POS tags (PRON, VERB, NOUN, DET)
- **Observations:** Words (eat, rice, dog, etc.)
- **Transition Probability:** Probability of moving from one tag to another.
- **Emission Probability:** Probability of a tag generating a particular word.

The **Viterbi Algorithm** is then used to determine the most likely sequence of POS tags.

Algorithm

1. Initialize the training dataset with tagged sentences.
2. Count the frequency of each POS tag.
3. Count the occurrence of every word for each tag (Emission Count).
4. Count transitions between consecutive tags (Transition Count).
5. Calculate Transition Probabilities.
6. Calculate Emission Probabilities.
7. Accept an input sentence from the user.
8. Split the sentence into individual words.
9. Initialize the Viterbi matrix using the first word.
10. Compute probabilities for all remaining words.
11. Store the highest probability and previous tag using the backpointer.
12. Trace back the best tag sequence.
13. Display the predicted POS tags.

Procedure

Step 1

Import the training dataset containing words and their POS tags.

Step 2

Initialize three dictionaries:

- transition_count
- emission_count
- tag_count

Step 3

Train the model by reading every sentence.

Step 4

Count every POS tag.

Step 5

Count every word belonging to each tag.

Step 6

Count transitions between tags.

Example

START → PRON

PRON → VERB

VERB → NOUN

Step 7

Calculate

Transition Probability

Step 8

Calculate

Emission Probability

Step 9

Accept the input sentence.

Example

I eat rice

Step 10

Apply the Viterbi Algorithm.

The algorithm calculates

Previous Probability

×

Transition Probability

×

Emission Probability

for every possible tag.

Step 11

Select the path having the highest probability.

Step 12

Display the predicted POS tags.

Code Explanation

```
main.py  [ ] [ ] [ ] Share Run

1 - train_data = [
2     [("I", "PRON"), ("eat", "VERB"), ("rice", "NOUN")],
3     [("She", "PRON"), ("likes", "VERB"), ("music", "NOUN")]
4     ,
5     [("The", "DET"), ("dog", "NOUN"), ("barks", "VERB")],
6     [("A", "DET"), ("cat", "NOUN"), ("sleeps", "VERB")]
7 ]
8 transition_count = {}
9 emission_count = {}
10 tag_count = {}
11 for sentence in train_data:
12     previous_tag = "START"
13     for word, tag in sentence:
14         word = word.lower()
15         if tag not in tag_count:
16             tag_count[tag] = 0
17         tag_count[tag] += 1
18         if tag not in emission_count:
19             emission_count[tag] = {}
20         if word not in emission_count[tag]:
21             emission_count[tag][word] = 0
22         emission_count[tag][word] += 1
23         if previous_tag not in transition_count:
24             transition_count[previous_tag] = {}
25         if tag not in transition_count[previous_tag]:
26             transition_count[previous_tag][tag] = 0
```

```
main.py  [ ] [ ] [ ] Share Run

28 - def transition_probability(prev_tag, curr_tag):
29     if prev_tag not in transition_count:
30         return 0.0001
31     total = sum(transition_count[prev_tag].values())
32     count = transition_count[prev_tag].get(curr_tag, 0)
33     if count == 0:
34         return 0.0001
35     return count / total
36
37
38 - def emission_probability(tag, word):
39     word = word.lower()
40
41     if tag not in emission_count:
42         return 0.0001
43
44     total = tag_count[tag]
45     count = emission_count[tag].get(word, 0)
46
47     if count == 0:
48         return 0.0001
49
50     return count / total
51
52 - def hmm_pos_tag(sentence):
```

```
main.py
86         best_score = score
87         best_previous = previous_tag
88
89         current_scores[current_tag] = best_score
90         current_back[current_tag] = best_previous
91
92         viterbi.append(current_scores)
93         backpointer.append(current_back)
94
95     best_tag = max(viterbi[-1], key=viterbi[-1].get)
96     result = [best_tag]
97
98     for i in range(len(words) - 1, 0, -1):
99         best_tag = backpointer[i][best_tag]
100         result.insert(0, best_tag)
101
102     return list(zip(words, result))
103
104 sentence = input("Enter a sentence: ")
105
106 result = hmm_pos_tag(sentence)
107
108 print("\nPredicted Tags:\n")
109
110 for word, tag in result:
111     print(word, "->", tag)
```

```
Output
Clear

Enter a sentence: i read books while going to bed

Predicted Tags:

i -> PRON
read -> VERB
books -> NOUN
while -> VERB
going -> NOUN
to -> VERB
bed -> NOUN

=== Code Execution Successful ===
```

Training Dataset

train_data

Contains labeled sentences used for training.

Example

I → PRON

eat → VERB

rice → NOUN

Transition Count

Stores

START $\rightarrow$ PRON

PRON $\rightarrow$ VERB

VERB $\rightarrow$ NOUN

This is used to calculate transition probabilities.

Emission Count

Stores

VERB

eat = 1

likes = 1

barks = 1

sleeps = 1

This is used to calculate emission probabilities.

Tag Count

Stores

PRON = 2

DET = 2

VERB = 4

NOUN = 4

Transition Probability Function

Calculates

$P(\text{Current Tag} \mid \text{Previous Tag})$

Formula

Emission Probability Function

Calculates

$P(\text{Word} \mid \text{Tag})$

Formula

Viterbi Algorithm

The Viterbi Algorithm calculates the probability of every possible tag for each word and selects the most probable sequence.

It consists of three stages:

- Initialization
- Recursion
- Backtracking

Result

The Hidden Markov Model (HMM) for Part-of-Speech (POS) Tagging was successfully implemented using the Viterbi Algorithm. The model learned transition and emission probabilities from the training dataset and correctly predicted the grammatical categories of words in the test sentences. The experiment demonstrates the effectiveness of probabilistic sequence modeling for NLP tasks and provides a strong foundation for understanding more advanced tagging techniques.